Escuela Da Vinci

Carrera Analista de Sistemas · Seminario Final

GoTrain

Aplicación social de fitness con Coach IA

Diseño de Arquitectura

Alumno: Santiago Ghiano

Versión 2.0 · 11/11/2026

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	24/09/2026	Santiago Ghiano	Versión inicial (Parcial 1, sobre prototipos).
2.0	11/11/2026	Santiago Ghiano	Actualización según la implementación final (Parcial 2).

Índice

Control de versiones 2
Índice 2
1. Visión general 3
2. Capas 4
3. Decisiones de diseño 4
4. API REST 4
5. Flujo del Coach IA 5
6. Seguridad 5
7. Despliegue 6

1. Visión general

GoTrain usa una arquitectura **cliente-servidor en tres capas**. Dos clientes (la aplicación web y la app mobile Android) consumen la misma **API REST**. La API concentra la lógica de negocio y el acceso a datos, y se integra con la **Claude API** de Anthropic para el Coach virtual.

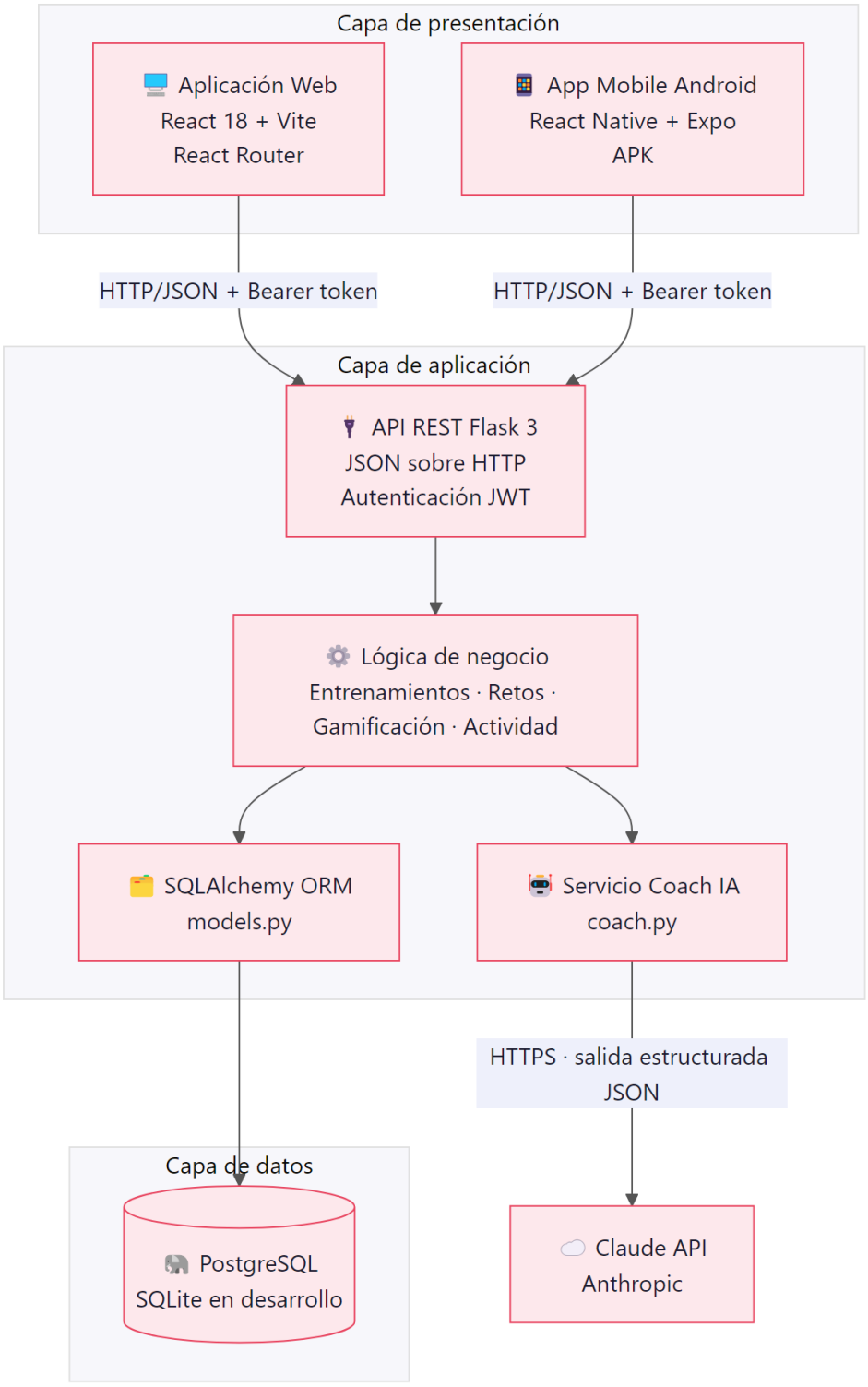


Figura 1 — Arquitectura en capas de GoTrain

2. Capas

Capa	Tecnología	Responsabilidad
Presentación — Web	React 18, React Router 6, Vite 6, CSS propio	SPA con landing, login/registro y panel. Guarda el token en localStorage.
Presentación — Mobile	React Native 0.88, Expo SDK 58, AsyncStorage	App Android (APK) con navegación por pestañas. Guarda token y servidor en el dispositivo.
Aplicación	Python 3.13, Flask 3, Flask-CORS, PyJWT	API REST, autenticación, validaciones, reglas de negocio (XP, racha, retos, logros).
Servicio IA	SDK anthropic (Python), Claude API	Genera respuestas y rutinas con salida estructurada JSON; modo demo como alternativa.
Datos	SQLAlchemy 2 + PostgreSQL 17 / SQLite	Persistencia. El motor se elige con DATABASE_URL.

3. Decisiones de diseño

- **Una sola API para web y mobile:** evita duplicar lógica y asegura que los datos sean los mismos en ambas plataformas.
- **JWT sin estado:** el servidor no guarda sesiones, lo que simplifica escalar horizontalmente.
- **Lógica de gamificación en el servidor:** XP, racha, progreso de retos y logros se calculan al guardar un entrenamiento, así los clientes no pueden alterarlos.
- **Salida estructurada de la IA:** se le pide a Claude un JSON con un esquema fijo (mensaje + rutina), lo que permite mostrar e importar la rutina de forma confiable.
- **Degradación controlada:** si la IA falla, el Coach usa reglas (modo demo) y la app sigue funcionando.
- **ORM:** el mismo código corre sobre SQLite (desarrollo y pruebas) y PostgreSQL (producción).

4. API REST

Formato JSON. Autenticación con el encabezado **Authorization: Bearer <token>**. Errores con el formato **{"error": "mensaje"}** y códigos 400 / 401 / 404.

Método	Ruta	Token	Descripción
POST	/api/auth/register	No	Registro
POST	/api/auth/login	No	Login, devuelve token
GET / PUT	/api/me	Sí	Datos del usuario / editar perfil
GET	/api/profile	Sí	Perfil con stats, logros e historial
GET	/api/dashboard	Sí	Panel de inicio
GET	/api/search?q=	Sí	Búsqueda
GET	/api/workouts	Sí	Historial
GET	/api/workouts/today	Sí	Plan de hoy
POST	/api/workouts/{id}/exercises	Sí	Agregar ejercicio
PUT / DELETE	/api/exercises/{id}	Sí	Editar / quitar ejercicio
POST	/api/workouts/{id}/start	Sí	Iniciar cronómetro
POST	/api/workouts/{id}/finish	Sí	Finalizar y guardar

Método	Ruta	Token	Descripción
POST	/api/workouts/running	Sí	Registrar running
GET	/api/routines	Sí	Rutinas guardadas
POST	/api/routines/{id}/use	Sí	Usar rutina
GET	/api/challenges?tab=	Sí	Retos por pestaña
POST / DELETE	/api/challenges/{id}/join	Sí	Unirse / salir
GET	/api/activity	Sí	Feed de actividad
POST	/api/activity/{id}/like	Sí	Felicitar / quitar
GET / POST	/api/coach/messages	Sí	Chat con el Coach
POST	/api/coach/messages/{id}/import	Sí	Importar rutina
GET	/api/health	No	Estado del servicio

5. Flujo del Coach IA

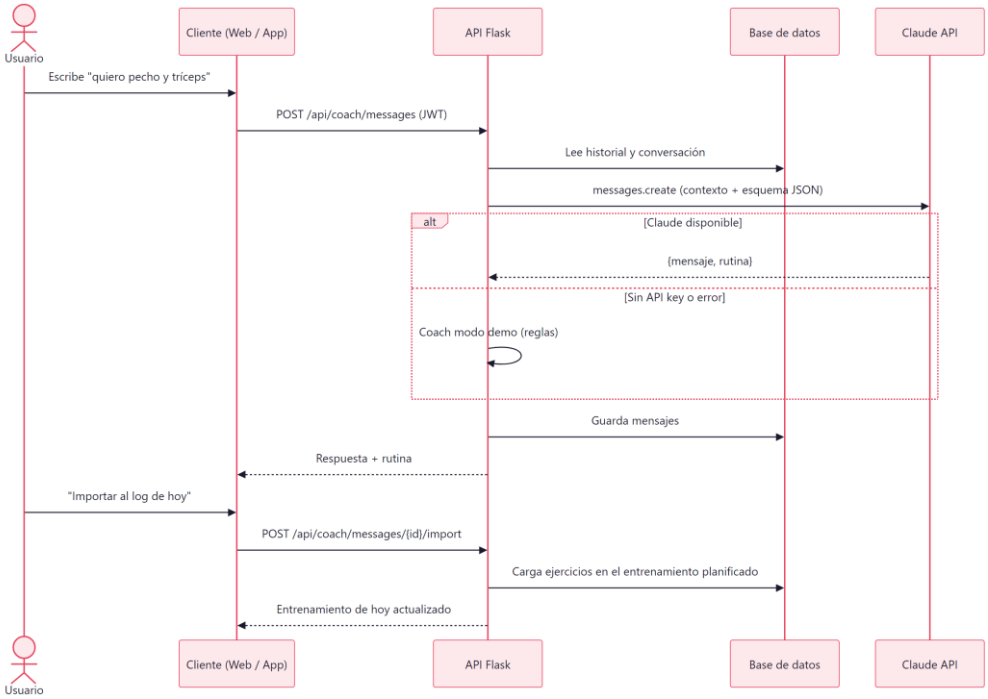


Figura 2 — Secuencia de una consulta al Coach

6. Seguridad

- Contraseñas con hash (Werkzeug scrypt).
- Tokens JWT HS256 con vencimiento de 7 días.
- Cada consulta filtra por el usuario del token (no se accede a datos ajenos).
- Secretos (SECRET_KEY, ANTHROPIC_API_KEY) en variables de entorno.
- CORS habilitado para los clientes; en producción se recomienda HTTPS.

7. Despliegue

Elemento	Desarrollo local	Publicación (Render, render.yaml)
API Flask	python app.py (puerto 5000)	Servicio web "gotrain-api" con Gunicorn y HTTPS
Base de datos	SQLite (backend/instance/gotrain.db)	PostgreSQL administrado "gotrain-db"
Web	Vite dev server (puerto 5173, proxy /api)	Sitio estático "gotrain-web" (npm run build) con VITE_API_URL
Mobile	Expo Go en la misma red WiFi	APK release (arm64) apuntando a la API publicada